

# Sisteme de Operare

## Cap 9. Alocarea memoriei (I)

May 4, 2023

- 1 Fundamente
- 2 Alocarea contiguă a memoriei
- 3 Paginarea
- 4 Structura tabeli de pagini
- 5 Swapping
- 6 Exemple de arhitecturi HW

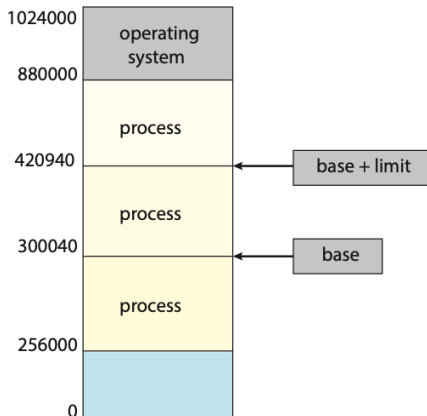
- 1 Fundamente
- 2 Alocarea contiguă a memoriei
- 3 Paginarea
- 4 Structura tabeli de pagini
- 5 Swapping
- 6 Exemple de arhitecturi HW

# Perspectiva hardware a memoriei

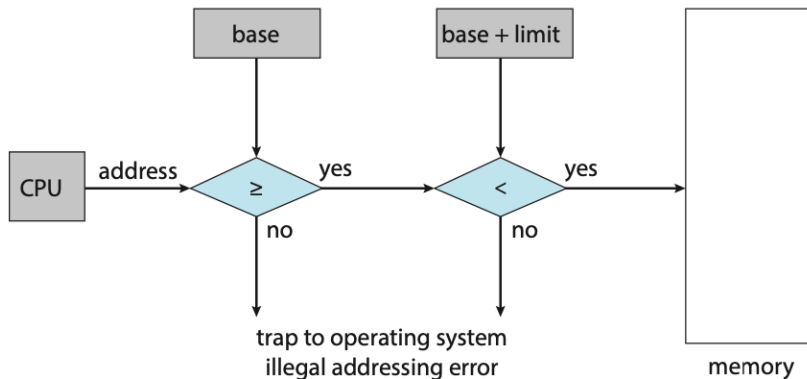
- memoria: array cuprinzător de bytes, fiecare identificat de o adresă proprie
- CPU-ul aduce instrucțiuni din memorie în concordanță cu registrul PC; acestea pot provoca operații suplimentare de tip LOAD/STORE
- un ciclu al unei instrucțiuni cuprinde: aducerea din memorie, decodarea ei ce poate determina aducerea de operanzi suplimentari din memorie, execuția sa și stocarea rezultatelor în memorie
- memoria principală împreună cu regiștrii procesor sunt singurele suporturi de stocare pe care CPU-ul le poate accesa direct; există instrucțiuni ce folosesc adrese de memorie dar nu există instrucțiuni ce pot accesa adrese de pe SSD, de exemplu; dacă datele nu sunt deja în memorie ele trebuie stocate acolo ca CPU-ul să le poată folosi
- regiștrii sunt parte din CPU și pot fi accesați într-un singur ciclu de ceas; aducerea datelor din memoria externă poate dura mai mulți cicli de ceas

## Perspectiva hardware a memoriei (2)

- procesorul nu lucrează cât timp vin datele din memorie; pentru remediere, se adaugă **cache**, o memorie rapidă care lucrează independent (fără controlul SO)
- protecția zonei de memorie a unui proces trebuie realizată de hardware; fiecare proces va avea propriul spațiu de memorie

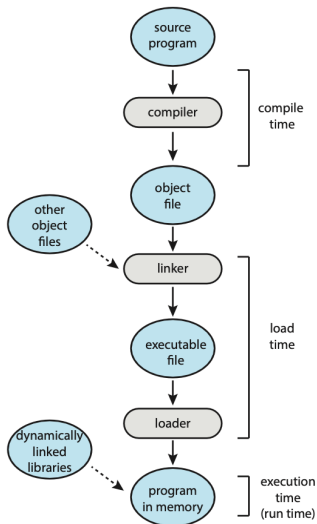


# Schemă simplă de protecție hardware a memoriei



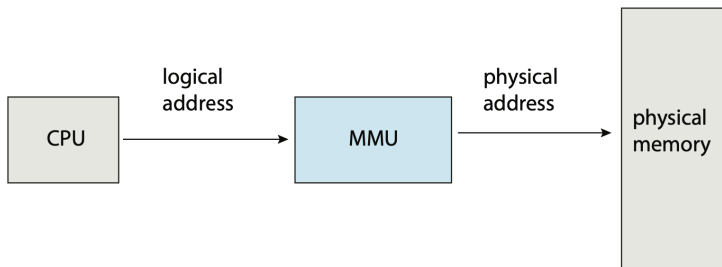
- orice încercare a unui program de a accesa memorie în afara [base, base + limit] duce la **memory trap**; SO are însă acces nelimitat
- modificarea regiștrilor de **base** și **limit** se face cu instrucțiuni privilegiate (kernel mode)

# Legarea adreselor



- dacă la momentul **compilării** se cunoaște adresa de memorie unde programul va fi încărcat, se poate genera **cod absolut**, adresele de memorie folosite de program nu se mai schimbă
- altfel compilatorul generează **cod relocabil**, iar adresele folosite de program se calculează la încărcare
- dacă programul poate fi mutat în timpul rulării în alt segment de memorie, calculul adreselor se amână pentru momentul **execuției** - abordarea uzuală

# Spațiul de adrese logic vs. spațiul de adrese fizic



- adresa generată de CPU este de obicei o **adresă logică** (sau adresă virtuală) vs. **adresă fizică** generată de Memory Management Unit (MMU)
- pentru schema simplă, adresa de bază (de ex. 14000) este adunată la fiecare adresă virtuală (ex. adresa virtuală 130 → adresa fizică 14130)
- adresele logice sunt între  $[0, \text{max}]$  iar cele fizice între  $[R, R + \text{max}]$ , pentru **base** = R



# Încărcarea dinamică (dynamic loading)

- am presupus că întreg programul e încărcat de la început în memorie
- cu **încărcarea dinamică** o procedură nu este încărcată decât înainte de a fi chemată
- încărcarea metodei se face de către încărcătorul de cod relocabil al SO; acesta ajustează codul relocabil al programului la adresa zonei de memorie unde este încărcat
- avantajul este folosirea eficientă a memoriei

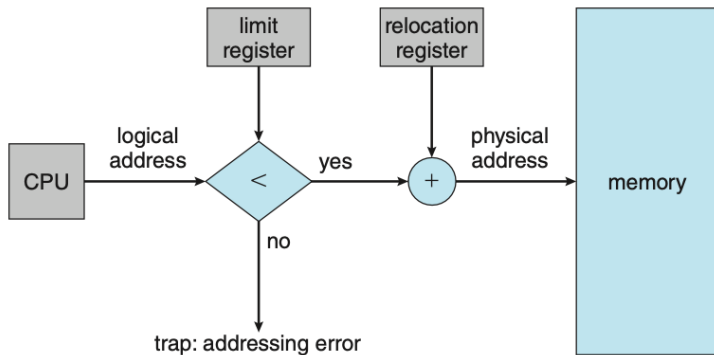
# Legarea dinamică și bibliotecile partajate

- bibliotecile legate dinamic (DLL) sunt componente SO legate la program când acesta rulează
- **bibliotecile statice** conțin cod care este legat la codul programului executabil la momentul legării
- DLL-urile reduc dimensiunea programului și permit reutilizarea codului
- DLL-ul va fi încărcat de SO doar dacă este nevoie de o funcție din el, iar atunci codul este relocat corespunzător (adresele folosite de funcție sunt recalculate)
- dacă un DLL este folosit de programe diferite, aflate în segmente de memorie diferite, SO este singura componentă ce permite accesul simultan a mai multor procese la același spațiu de adrese

- 1 Fundamente
- 2 Alocarea contiguă a memoriei
- 3 Paginarea
- 4 Structura tabeli de pagini
- 5 Swapping
- 6 Exemple de arhitecturi HW

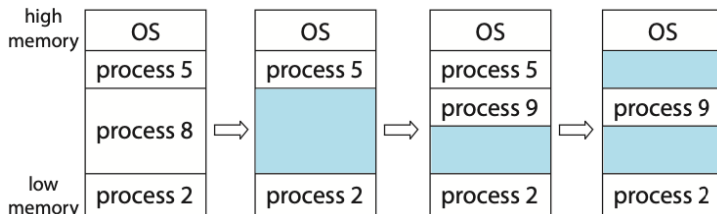
## Protecția memoriei

- se concentrează pe alocarea eficientă a memoriei, care stochează SO și cât mai multe procese
- alocarea contiguă: fiecare proces este stocat într-o singură secțiune de memorie adiacentă cu cea a următorului proces
- schema de relocare permite SO modificarea dinamică a mărimii sale



# Alocarea memoriei

- cea mai simplă: fiecărui proces i se asociază o partiție de mărime variabilă
- ca urmare a realocărilor memoria ajunge să se fragmenteze, va conține mai multe **găuri**
- la crearea unui proces nou, se caută o **gaură** (hole) suficient de mare cât să se încadreze
- **problema alocării dinamice** a memoriei are mai multe strategii de rezolvare: first fit, best fit, worst fit



# Strategii de alocare dinamică

- **first fit**: se alocă primul spațiu liber suficient de încăpător; căutarea poate începe de la început de fiecare dată sau de la ultima poziție găsită
- **best fit**: cel mai mic spațiu suficient de încăpător; se caută în întreaga listă (timp liniar), doar dacă lista nu e menținută sortată (heap); produce cele mai mici spații libere rămase
- **worst fit**: alocă cel mai mare spațiu liber; presupune căutarea în toată lista; produce cele mai mari găuri rămase
- simulările arată că primele două variante sunt mai eficiente atât ca timp cât și ca satisfacere a cererii de alocare; nu există diferență clară d.p.d.v. al satisfacerii cererii de stocare, dar first fit este mai rapid

# Fragmentarea

- strategiile de alocare produc **fragmentare externă** (găuri prea mici, nealocabile)
- **regula 50%**: analiza statistică arată că pentru best fit și first fit, la  $N$  blocuri alocate sunt alte  $N/2$  blocuri pierdute datorită fragmentării - o treime din memorie devine neutilizabilă
- o soluție la problema fragmentării externe este **compactarea**; dacă relocarea este dinamică, se poate face compactare prin mutarea blocurilor în memorie și relocarea codului
- o altă soluție este permiterea spațiilor logice de adrese să fie necontigue, folosind **paginarea**; este cea mai utilizată tehnică ce elimină problema fragmentării

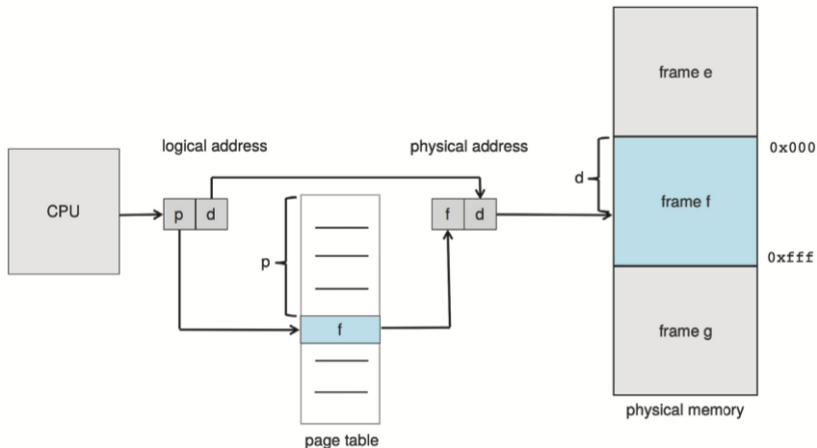
- 1 Fundamente
- 2 Alocarea contiguă a memoriei
- 3 Paginarea**
- 4 Structura tabeli de pagini
- 5 Swapping
- 6 Exemple de arhitecturi HW



# Metoda de bază

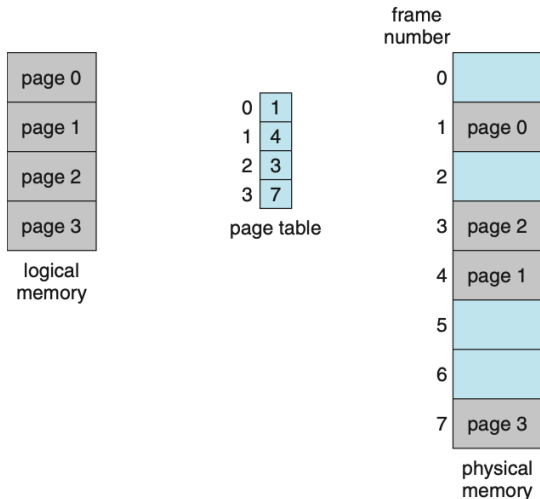
- **paginarea** permite ca spațiul de adrese fizice al unui proces să nu fie contiguu
- paginarea previne apariția fragmentării și necesitatea compactării
- folosită în majoritatea SO prin colaborarea dintre SO și HW
- metoda de bază pp. împărțirea memoriei fizice în **frame-uri**, blocuri de mărime fixă; memoria logică se împarte tot în blocuri de mărime fixă denumite **pagini**; la execuție, paginile sale sunt încărcate în frame-urile disponibile
- spațiul de adrese logice este complet separat de spațiul fizic de adrese; un proces poate avea 64 de biți de adresă deși memoria fizică are mai puțin de  $2^{64}$  bytes
- fiecare adresă logică (folosită de CPU) este împărțită în **page number** (p) și **page offset**, displacement (d)

# HW pentru paginare



- page number este folosit ca index în **page table**, tabelă specifică fiecărui proces; MMU face translatarea din număr de pagină în adresă fizică, folosind frame-ul asociat paginii

# Exemplu de paginare (1)

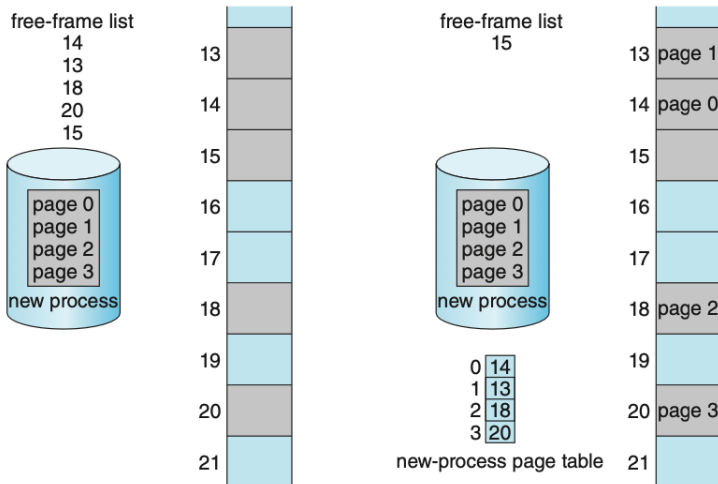


- mărimea paginii (și implicit a frame-ului) este definită ca putere a lui 2, între 4KB și 1GB

## Exemplu de paginare (2)

- adresa de memorie calculată de exemplul anterior are 2 biți pentru pagină (frame) și 2 biți pentru offset
- pagina va avea 4 bytes ( $2^2$ ) iar memoria totală are 32 bytes ( $4 \times 8$  frame-uri)
- adresa logică 0 este în pagina 0, frame 1,  $AF = 4 \times 1 + 0 = 4$
- adresa logică 3 este în pagina 0, frame 1,  $AF = 4 \times 1 + 3 = 7$
- adresa logică 6 este în pagina 1, frame 4,  $AF = 4 \times 4 + 2 = 18$
- adresa logică 9 este în pagina 2, frame 3,  $AF = 4 \times 3 + 1 = 13$
- adresa logică 14 este în pagina 3, frame 7,  $AF = 4 \times 7 + 2 = 30$
- putem avea fragmentare internă, ex. un proces ce necesită 72.766 bytes, pentru pagini de 2.048 bytes, necesită 35 de pagini și 1.086 bytes; i se alocă 36 frames, iar fragmentarea internă duce la  $2.048 - 1.086 = 962$  bytes nefolosiți dar alocați
- fragmentarea internă așteptată e de o jumătate de pagină per proces  
→ dimensiune mică a paginii; overhead → dimensiune mare

# Exemplu de funcționare a MMU



- frame-urile disponibile înainte de alocare (14, 13, 18, 20, 15) și după alocare (15)

## HW pentru paginare

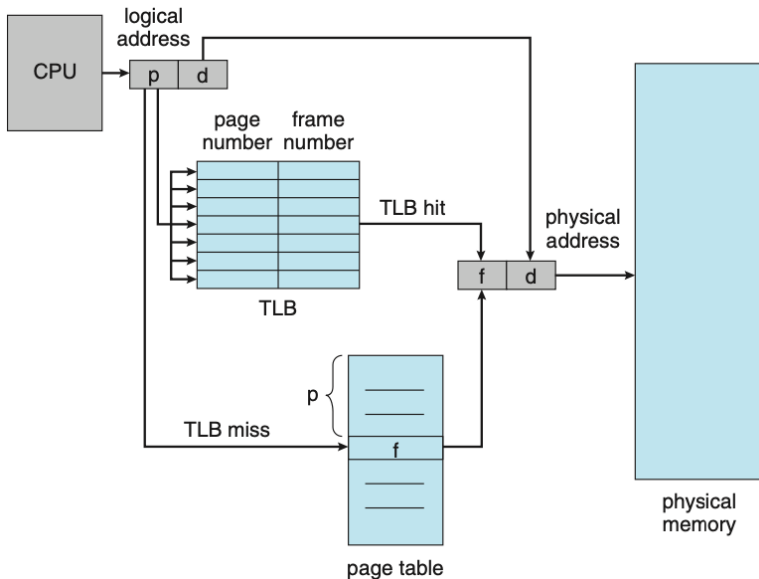
- maparea paginilor în frame-uri este ascunsă prin HW de translatare
- adresele logice văzute de utilizator sunt consecutive, dar în realitate adresele fizice sunt "împrăștiate" în memoria fizică
- fiecare proces are asociat această tabelă de pagini (**page table**); SO menține o tabelă de frame-uri (**frame table**) pentru evidența alocării
- implementarea simplă folosește un set de regiștri rapizi, dedicați, pentru stocarea tabelii de pagini; dezavantajul este creșterea timpului de context switch, deoarece și acești regiștri trebuie salvați / restaurați
- soluția cu regiștri are sens doar pentru tabele de pagini de dimensiuni reduse (până în 256 intrări)
- tabelele de pagini moderne au de regulă  $2^{20}$  intrări, ele sunt stocate în memoria principală, există un **page-table base register** (PTBR) care indică spre începutul tabelii; avantaj: context switch time redus; dezavantaj: fiecare acces la memorie ascunde în spate de fapt două accese (timp crescut de acces la memorie)

# Translation Look-aside Buffer (TLB)

- este o memorie asociativă rapidă<sup>1</sup>
- fiecare intrare constă dintr-o pereche de forma (cheie, valoare)
- când TLB-ului i se prezintă un item, acesta este comparat simultan cu toate cheile; se întoarce valoarea corespunzătoare cheii care coincide
- TLB-ul este rapid, este parte a pipeline-ului de execuție a instrucțiunilor
- TLB conține doar puține intrări, ex. 32 - 1.024
- funcționare:
  - la folosirea unei adrese logice de către CPU, MMU verifică dacă numărul de pagină este prezent în TLB; dacă da, numărul de frame este disponibil imediat
  - dacă numărul de pagină nu este în TLB (**TLB miss**), se caută în memorie numărul frame-ului și se actualizează TLB-ul
  - dacă TLB-ul este deja plin, se eliberează una, fie prin strategia Least Recently Used (LRU) fie round-robin
  - în mod tipic, anumite intrări în TLB care corespund zonelor kernel nu pot fi eliminate (sunt **wired down**)

<sup>1</sup>[https://en.wikipedia.org/wiki/Content-addressable\\_memory](https://en.wikipedia.org/wiki/Content-addressable_memory)

## HW de paginare cu TLB





## Funcționarea TLB-ului

- unele TLB-uri permit stocarea unui **address-space identifier** (ASID) în fiecare intrare; el identifică unic un proces
- dacă identificatorul procesului curent nu corespunde cu identificatorul paginii virtuale, operația rezultă într-un TLB miss
- pe lângă rolul de protecție al spațiului de adrese, ASID-ul permite prezența intrărilor aparținând mai multor procese diferite, în mod simultan
- dacă TLB-ul nu suportă ASID, ea trebuie golită (flushed) la fiecare context switch
- procentul numărului de ori pentru care o intrare căutată este găsită ca deja fiind în TLB se numește **hit ratio**
- de ex. dacă intrarea este în TLB, accesul la memorie costă 10 ns; dacă nu există, avem nevoie să accesăm tabela de pagini (încă 10 ns), în total accesul la acea locație costă 20 ns (în cazul fericit)
- timpul efectiv de acces pentru 80% hit ratio (rata realistă este 99%):

$$\text{effective access time} = 0.80 \times 10 + 0.20 \times 20 = 12[\text{ns}]$$

# Protecția

- protecția este realizată prin verificarea biților de protecție asociați cu fiecare frame, biți stocați în tabela de pagini
- o încercare de scriere într-o pagină pentru care bitul de protecție este read-only determină un trap - memory protection violation
- SO asociază un alt bit, denumit **valid-invalid** bit, care indică dacă pagina este în spațiul logic de adrese al procesului
- adresele ilegale sunt detectate de hardware și transmise SO (trap)
- de regulă un proces nu folosește întreg spațiul de adrese; ca atare, tabela de pagini poate fi mai scurtă, ca să nu ocupe memorie în mod inutil; anumite sisteme au un registru, **page-table length register** (PTLR) ce indică lungimea tabelului de pagini; acest registru este verificat pentru fiecare adresă logică, iar în caz că adresa depășește valoarea din registru, HW generează o întrerupere care este trimisă SO ca system trap

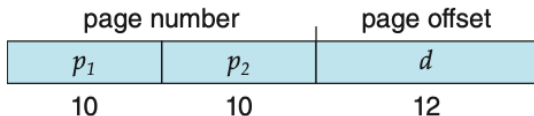
## Pagini partajate (shared)

- ex. biblioteca sistem libc poate fi folosită de mai multe procese, se evită necesitatea de a încărca biblioteca în spațiul de adrese al fiecărui proces
- pentru a putea fi chemat din mai multe procese simultan, codul bibliotecii trebuie să fie **reentrant**, adică să nu se auto-modifice; fiecare proces are propria copie a setului de regiștri și date
- paginile în care se încarcă codul reentrant sunt marcate ca read-only; securizarea nu se lasă la latitudinea programului ci este verificată prin HW
- paginile care implementează comunicații inter-procese (IPC) prin memorie partajată nu sunt marcate ca read-only

- 1 Fundamente
- 2 Alocarea contiguă a memoriei
- 3 Paginarea
- 4 Structura tabeli de pagini**
- 5 Swapping
- 6 Exemple de arhitecturi HW

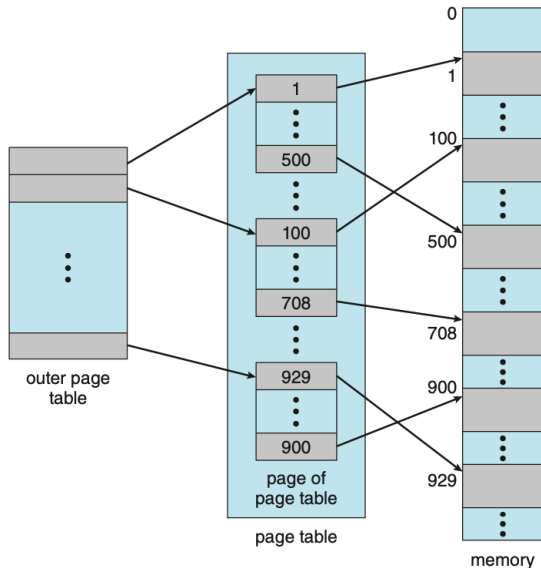
## Paginarea ierarhică

- calculatoarele moderne suportă spații de adrese logice de  $2^{32}$  sau  $2^{64}$  adrese
- dacă dimensiunea paginii este 4KB (adresabilă cu 12 biți), pentru un spațiu de  $2^{32}$  adrese vom avea  $2^{20}$  intrări în tabela de pagini, cam 1 milion de intrări
- dacă fiecare intrare are 4 bytes, fiecare tabelă de pagini per proces ocupă 4MB; la 50 de procese (tipic), 200 MB sunt ocupați doar de tabelele de pagini - mult!
- tabela de pagini se împarte în bucăți, în care tabela de pagini este ea însăși paginată
- adresa logică se împarte în:



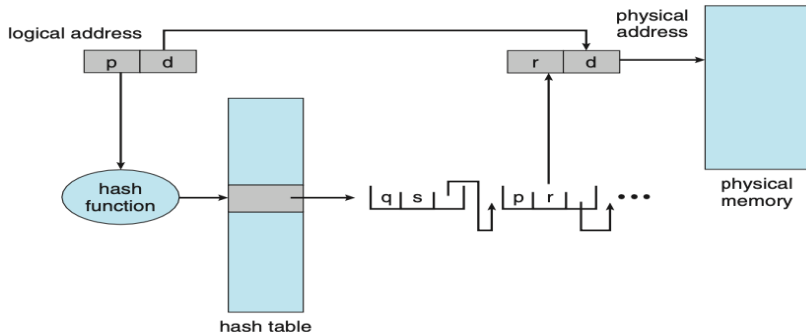
# Forward-mapped page table

- $p_1$  indică intrarea din outer page table (din cele 1.024)
- nu toate intrările din outer page table trebuie să conțină adrese de tabele de pagini
- pentru un sistem cu 64 biți de adresă și pagini de 4KB, tabela de pagini are  $2^{52}$  intrări; outer page table ajunge la  $2^{42}$  intrări, adică  $2^{44}$  bytes - mai multe nivele



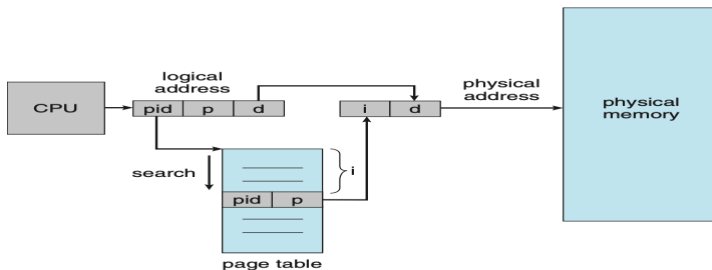
## Hashed page table

- folosită pentru a adapta spațiu de adrese mai mare de 32 de biți
- fiecare intrare în hash table este o listă înlănțuită a coliziunilor
- numărul de pagină este folosit pentru identificarea listei, urmat de căutarea elementului în listă
- o variațiune este **clustered page tables**, folosit la spații de adrese de 64 biți, în care lista are exact 16 intrări; utile pentru spații de adrese **sparse**, unde referințele la memorie sunt necontigue și împrăștiate



# Inverted page tables

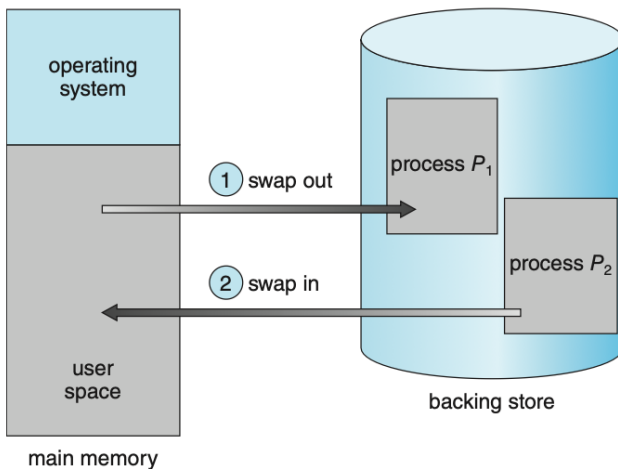
- tabelele de pagini obișnuite folosesc biții spațiului de adrese logice pentru a căuta în tabelă frame-ul corespondent; se poate ajunge la un număr prohibitiv de intrări în tabelă, sau de folosirea de tabele multi-nivel, ce cresc timpul de acces la memorie
- soluția sunt **tabele de pagini inversate**, ce conțin o intrare pentru fiecare frame (pagină reală) de memorie folosită
- fiecare intrare conține adresa virtuală a paginii stocate în memoria reală, împreună cu ID-ul de proces





- 1 Fundamente
- 2 Alocarea contiguă a memoriei
- 3 Paginarea
- 4 Structura tabeli de pagini
- 5 Swapping**
- 6 Exemple de arhitecturi HW

# Definiția swapping-ului



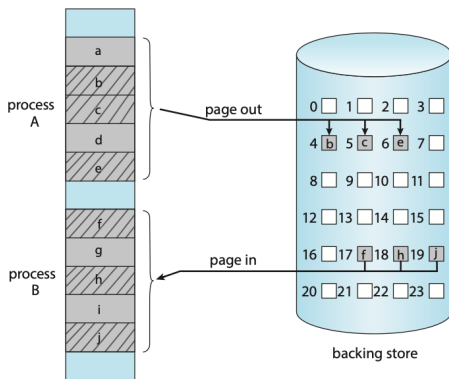
- un proces sau o porțiune a sa poate fi **swapped out** din memorie spre un dispozitiv de stocare, iar ulterior adus înapoi și execuția reluată

# Swapping standard

- swapping-ul face posibil faptul ca spațiul total fizic de adrese al proceselor să depășească memoria fizică reală prezentă în sistem, crescând gradul de multiprogramare
- metoda standard presupune swapping-ul întregului proces, cu toate structurile sale de date, inclusiv thread-urile aferente
- stocarea se face pe dispozitivul secundar de stocare (primul este memoria RAM)
- candidați pentru swapping sunt procesele idle sau aproape idle
- un proces swapped out care redevine activ va fi swapped in (restaurat) de către SO
- apelarea frecventă de SO a swapping-ului este un indiciu că sunt mai multe procese active decât suportă memoria sistemului; soluțiile pot fi: (1) reducerea numărului de procese sau (2) mărirea memoriei RAM a sistemului

# Swapping cu paginare

- swapping-ul proceselor întregi nu se mai folosește în SO moderne (durează prea mult)
- mai degrabă sunt swapped pagini ale procesului
- termenul de **swapping** este folosit în sens generic, pe când **paging** (paginarea) se referă la procesul de swapping al paginilor (virtuale) de memorie

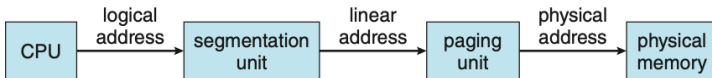


# Swapping în sisteme mobile

- de regulă evită swapping-ul pentru că memoria secundară este memorie flash, limitată ca mărime
- alt motiv este numărul limitat de scrieri suportat de memoria flash, memoria "se uzează" mai repede dacă există un mecanism de swapping activ, respectiv viteză de transfer limitată din rațiuni de consum
- Apple iOS nu folosește swapping ci **roagă** aplicațiile să redea memoria alocată, de ex. datele read-only precum codul sunt eliminate din memorie și readuse când este necesar
- aplicațiile care nu se conformează sunt eliminate (întrerupte)
- Android adoptă o strategie similară, poate termina o aplicație dacă memoria devine insuficientă; înainte de terminare îi scrie starea în memoria flash pentru a putea fi restaurată rapid

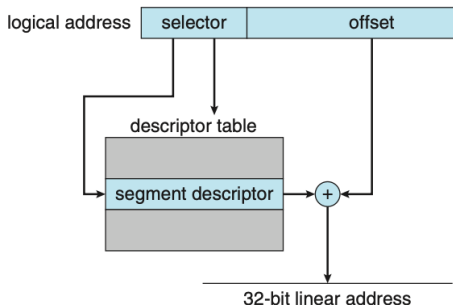
- 1 Fundamente
- 2 Alocarea contiguă a memoriei
- 3 Paginarea
- 4 Structura tabeli de pagini
- 5 Swapping
- 6 Exemple de arhitecturi HW**

# Arhitectura IA-32



- sunt două unități, segmentarea, ce generează o adresă liniară pentru fiecare adresă logică, pe baza căreia paginarea generează adresa fizică
- un segment poate avea dimensiunea maximă de 4GB, numărul maxim de segmente per proces este 16K
- spațiul logic de adrese se împarte în două partiții, 8K segmente private procesului și 8K segmente ce pot fi partajate cu alte procese
- informațiile despre prima partiție se păstrează în **Local Descriptor Table** (LDT), iar cea de-a doua, **Global Descriptor Table** (GDT); fiecare intrare este un descriptor de segment de 8 bytes

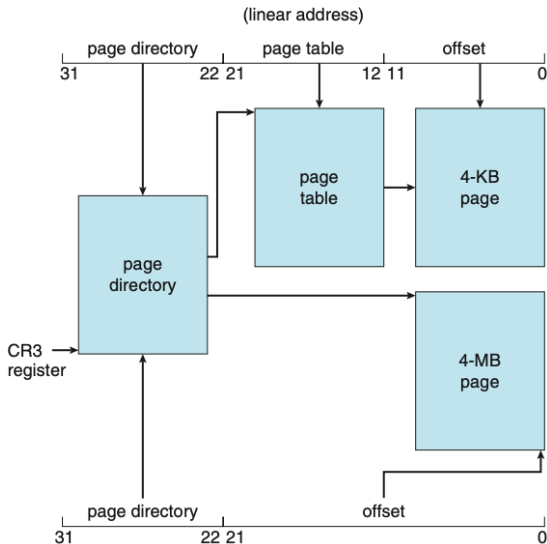
# Arhitectura IA-32 - segmentarea



- procesorul are 6 regiștri de segment, deci procesul poate accesa concomitent 6 segmente; descriptorii LDT și GDT sunt cache-uiți în 6 regiștri de microprogram de 8 bytes, reducând numărul de accese la LDT și GDT
- intrarea în LDT/GDT dată de selector determină adresa de bază respectiv limita (conținute în descriptorul de segment)
- se adună offset-ul la adresa de bază, determinînd adresa liniară

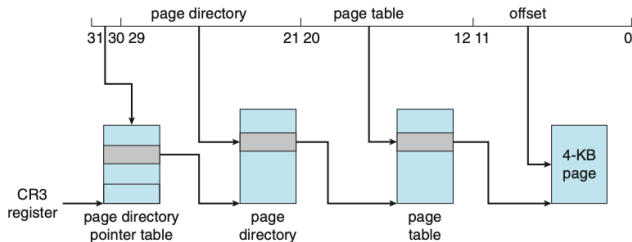


# Arhitectura IA-32 - paginarea



- schemă de paginare pe 2 nivele, pagini de 4KB / 4MB

# Page (Physical) Address Extension



- permite proceselor pe 32 de biți să acceseze mai mult de 4GB de memorie
- schema de paginare a trecut de la 2 la 3 nivele, unde most-significant 2 bits (MSB) referă o **page directory pointer table**
- PAE a mărit mărimea intrărilor din page directory respectiv page table de la 32 la 64 de biți, adresa de bază a page table / page frame a crescut de la 20 la 24 de biți ce suportă adresarea de până la 64 GB RAM
- OS trebuie să suporte extensia PAE oferită de HW (Win32 - 4GB)